

Kill the N+1 query



Get the same answers from the database in one trip instead of dozens.

The big idea

Each time Django asks the database for rows, that's one **query**: a round trip, like a trip to the shop. Buying 100 things in **one** trip is fine; **100 trips** is not.

The classic mistake is the **N+1 problem**: 1 query for a list of tasks, then 1 more **per task** to get its project. 500 tasks means 501 trips.

What the tests put in the database

Your functions take **no parameters**. Before each test, the tests create these rows:

```
website = Project.objects.create(name="Website")
api = Project.objects.create(name="API")
Project.objects.create(name="Zine")          # a project with no tasks

Task.objects.create(project=website, title="Design homepage", done=True)
Task.objects.create(project=api, title="Add auth")
Task.objects.create(project=website, title="Write copy")
Task.objects.create(project=api, title="Paginate", done=True)
Task.objects.create(project=website, title="Launch")
```

What your functions must return

The starter already gives these answers. Your job is to cut the **number of queries**.

```
task_labels()
# ["Website: Design homepage", "API: Add auth", "Website: Write copy",
#  "API: Paginate", "Website: Launch"]

list(project_summaries())
# [{"name": "API", "task_count": 2, "done_count": 1},
#  {"name": "Website", "task_count": 3, "done_count": 1},
#  {"name": "Zine", "task_count": 0, "done_count": 0}]

# Queries now: 6 and 7. Goal: exactly 1 each (tests 2, 4, 5)
```

New words

query

One question sent to the database and its answer: one round trip.

queryset

Django's description of a query. It runs when you loop or `list(...)` it.

N+1 problem

1 query for N things, then 1 more for **each** thing.

foreign key

A column pointing to a row in another table, like `task.project`.

JOIN

Combining two tables in one query, like `pd.merge`.

aggregate

Summarise many rows into one number: a count, a sum.

Tool: select_related

```
# uses the lesson's Garden / Kitchen sample data
with CaptureQueriesContext(connection) as queries:
    task = Task.objects.get(title="Bake bread")
    print(task.project.name)
print(len(queries), "queries")
with CaptureQueriesContext(connection) as queries:
    task = Task.objects.select_related("project").get(title="Bake bread")
    print(task.project.name)
print(len(queries), "queries")
```

OUTPUT

```
Kitchen
2 queries
Kitchen
1 queries
```

Tool: annotate with Count and Q

```
# the database counts each project's tasks, like groupby().size()
for project in Project.objects.annotate(n_tasks=Count("tasks")):
    print(project.name, project.n_tasks)

# count only the tasks that match a condition
for project in Project.objects.annotate(
    n_buy=Count("tasks", filter=Q(tasks__title__startswith="Buy"))
):
    print(project.name, project.n_buy)
```

OUTPUT

```
Garden 1
Kitchen 2
Garden 0
Kitchen 1
```

select_related vs annotate

select_related("project")

- Follows a foreign key: task to **its** project
- Like `pd.merge`: a JOIN
- Gives whole related objects
- You need it in `task_labels`

annotate(n=Count("tasks"))

- Counts the rows pointing **back**
- Like `groupby(...).size()`
- Gives one number per row
- You need it in `project_summaries`

Pattern: one chained queryset

Annotate, sort, then pick columns, like `df.to_dict("records")`:

```
rows = (
    Project.objects
    .annotate(n_tasks=Count("tasks"))
    .order_by("-name")
    .values("name", "n_tasks")
)
print(list(rows))
```

OUTPUT

```
[{'name': 'Kitchen', 'n_tasks': 2}, {'name': 'Garden', 'n_tasks': 1}]
```

Your task, step by step

- 1 Run the tests on the starter: answers pass, query counts fail → tests 1, 3
- 2 `task_labels`: add `.select_related("project")`, keep `.order_by("id")` → tests 2, 5
- 3 Replace the loop with `Project.objects.annotate(task_count=...)`
- 4 In the same `annotate`, add `done_count: Count` with `filter=Q(...)`
- 5 Sort by name with `.order_by("name")`: API, Website, Zine
- 6 Add `.values(...)` with the three fields, and return `list(...)` → tests 3-5

Common mistakes

- `.count()` or `.filter()` on `project.tasks` inside a loop: N+1 again
- Dropping `.order_by("id")`: labels come back in another order
- `select_related("projects")`: use the field name, `"project"`
- `tasks.done` or `tasks_done` in `Q(...)`: it's `tasks__done`
- `.values(...)` before `.annotate(...)`: `annotate` comes first

Where you'll use this

A DRF endpoint lists 50 tasks, and its serializer shows `task.project.name`. Every page load quietly runs 51 queries, and the API feels slow.

- Fix it in the view: `queryset = Task.objects.select_related("project")`
- Dashboards with counts per project use `annotate(Count(...))`
- For a project with all its tasks, use `prefetch_related("tasks")`: 2 queries

Check yourself

1. How many queries does a loop over 30 tasks printing `task.project.name` run?
2. When do you use `select_related`, and when `annotate`?
3. What does the double underscore in `tasks__done` mean?
4. Why let the database count instead of `len(project.tasks.all())`?

Keep learning

[How to Optimize Django REST APIs for Performance](#)

freeCodeCamp · article · beginner. Start here: explains the N+1 problem (100 posts, 101 queries) and how `select_related` and `prefetch_related` fix it.

[Database access optimization](#)

Django docs · docs · intermediate. Django's own checklist for fewer queries, including 'retrieve everything at once' and doing counting in the database.

[QuerySet API reference: select related](#)

Django docs · docs · intermediate. Reference for `select_related`, which joins Task and Project in one query for your task_labels report.

[Aggregation: filtering on annotations](#)

Django docs · docs · intermediate. Shows `annotate()` with `Count` and `Count(filter=Q(...))`, the tools behind your one-query project summary.